

NumPy & Matplotlib

A Detailed Function Reference

Each function is described, then every parameter is explained.

With signatures, defaults, return values, and short examples.

Companion reference for CSE 219 — Signals and Their Properties

Conventions: `np` = `numpy`, `plt` = `matplotlib.pyplot`, `ax` = an `Axes` object, `fig` = a `Figure` object.

Contents

1	NumPy	2
1.1	Array Creation and Initialization	2
1.2	Shape and Rearrangement	4
1.3	Indexing, Searching, Selection	7
1.4	Elementwise Math (Universal Functions)	8
1.5	Reductions and Statistics	10
1.6	Calculus and Signal Processing	12
1.7	Linear Algebra	13
1.8	Logic, Comparison, Complex	15
1.9	Random Sampling (Generator API)	15
2	Matplotlib	17
2.1	Figures and Axes	17
2.2	Plotting: Lines and Points	18
2.3	Plotting: Bars, Histograms, Images	19
2.4	Axis Configuration	21
2.5	Legends, Text, Annotations	23
2.6	Saving, Display, Global Style	23

1 NumPy

1.1 Array Creation and Initialization

Functions that build new ndarrays. Unless stated otherwise, shape may be an int (1-D) or a tuple of ints, and dtype defaults to float64 for the zeros/ones family.

```
np.array(object, dtype=None, copy=True, order='K', ndmin=0)
```

Create an ndarray from any array-like object (list, tuple, nested lists, or another array). **Parameters**

object *array-like* — Input data: a scalar, list, tuple, or nested sequence to convert.

dtype *data-type, optional* — Desired element type (e.g. np.float64, np.int32). If omitted it is inferred from the data.

copy *bool, default True* — If True always copy; if False avoid a copy when possible.

order {*'K', 'A', 'C', 'F'*}, *default 'K'* — Memory layout. 'C' is row-major, 'F' is column-major, 'K' keeps the input layout.

ndmin *int, default 0* — Minimum number of dimensions the result must have; ones are prepended to the shape as needed.

Returns: *ndarray* — The constructed array.

Example

```
a = np.array([[1, 2], [3, 4]], dtype=float)
```

```
np.asarray(a, dtype=None, order=None)
```

Convert the input to an ndarray, but do not copy if it is already an ndarray of the right dtype/order. Useful at the top of functions to accept lists or arrays uniformly. **Parameters**

a *array-like* — Input data to convert.

dtype *data-type, optional* — Target element type.

order {*'C', 'F', 'A', 'K'*}, *optional* — Desired memory layout.

Returns: *ndarray* — The input as an array (possibly the same object).

```
np.zeros(shape, dtype=float, order='C')
```

Return a new array of the given shape filled with zeros. **Parameters**

shape *int or tuple of ints* — Dimensions of the array, e.g. 5 or (3, 4).

dtype *data-type, default float64* — Element type of the array.

order {*'C', 'F'*}, *default 'C'* — Row-major (C) or column-major (F) storage.

Returns: *ndarray* — Array of zeros.

```
np.ones(shape, dtype=None, order='C')
```

Return a new array of the given shape filled with ones. Parameters mirror np.zeros. **Parameters**

shape *int or tuple of ints* — Dimensions of the array.

dtype *data-type, optional* — Element type (default float64).

order {*'C', 'F'*}, *default 'C'* — Memory layout.

Returns: *ndarray* — Array of ones.

```
np.full(shape, fill_value, dtype=None, order='C')
```

Return a new array of the given shape, every element set to fill_value. **Parameters**

shape *int or tuple of ints* — Dimensions of the array.

fill_value *scalar or array-like* — The value placed in every element.

dtype *data-type, optional* — Element type; inferred from fill_value if omitted.
order `{'C', 'F'}`, *default 'C'* — Memory layout.
Returns: *ndarray* — Constant-filled array.

np.empty(shape, dtype=float, order='C')

Allocate an array without initialising its entries. Faster than zeros/ones but the contents are arbitrary garbage until written. **Parameters**

shape *int or tuple of ints* — Dimensions of the array.
dtype *data-type, default float64* — Element type.
order `{'C', 'F'}`, *default 'C'* — Memory layout.

Returns: *ndarray* — Uninitialised array.

np.zeros_like(a, dtype=None, order='K', subok=True, shape=None)

Return an array of zeros with the same shape and dtype as a. The sibling functions ones_like, full_like, and empty_like behave identically for their respective fills. **Parameters**

a *array-like* — Prototype array whose shape and dtype are copied.
dtype *data-type, optional* — Override the prototype's element type.
order `{'C', 'F', 'A', 'K'}`, *default 'K'* — Memory layout of the result.
shape *int or tuple, optional* — Override the prototype's shape.

Returns: *ndarray* — Zeros matching a.

np.arange([start,] stop[, step[, dtype=None])

Return evenly spaced integer- or float-valued samples over the half-open interval [start, stop). Like Python's range but returns an array and accepts a float step. **Parameters**

start *number, optional* — First value (default 0).
stop *number* — End of the interval, exclusive.
step *number, optional* — Spacing between values (default 1). Beware of float rounding at the endpoint.
dtype *data-type, optional* — Output element type; inferred if omitted.

Returns: *ndarray* — 1-D array of samples.

Note: For a guaranteed number of points including the endpoint, prefer np.linspace.

np.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0)

Return num evenly spaced samples over the closed interval [start, stop]. The workhorse for building a continuous-time axis. **Parameters**

start *number* — First sample value.
stop *number* — Last sample value (included unless endpoint=False).
num *int, default 50* — Number of samples to generate.
endpoint *bool, default True* — If True, stop is the last sample; if False it is excluded.
retstep *bool, default False* — If True, also return the spacing between samples.
dtype *data-type, optional* — Output element type.
axis *int, default 0* — Axis along which the samples are stored when start/stop are arrays.

Returns: *ndarray (and float if retstep)* — The samples, and the step if requested.

Example

```
t = np.linspace(-5, 5, 1001)  # 1001 points, step 0.01
```

```
np.logspace(start, stop, num=50, endpoint=True, base=10.0, dtype=None, axis=0)
```

Return num samples spaced evenly on a log scale, i.e. $\text{base}^{\text{start}}$.. $\text{base}^{\text{stop}}$. **Parameters**

start *number* — $\text{base}^{\text{start}}$ is the first value.

stop *number* — $\text{base}^{\text{stop}}$ is the last value.

num *int, default 50* — Number of samples.

endpoint *bool, default True* — Include the stop value.

base *float, default 10.0* — Base of the log spacing.

dtype *data-type, optional* — Output element type.

Returns: *ndarray* — Log-spaced samples.

```
np.eye(N, M=None, k=0, dtype=float, order='C')
```

Return a 2-D array with ones on one diagonal and zeros elsewhere. **Parameters**

N *int* — Number of rows.

M *int, optional* — Number of columns (defaults to N).

k *int, default 0* — Diagonal index: 0 is the main diagonal, positive is upper, negative is lower.

dtype *data-type, default float* — Element type.

Returns: *ndarray* — The (N, M) matrix.

```
np.meshgrid(*xi, indexing='xy', sparse=False, copy=True)
```

From N 1-D coordinate vectors, produce N-D coordinate grids suitable for evaluating functions of several variables (e.g. surfaces, images). **Parameters**

xi *1-D arrays* — One or more coordinate vectors (x, y, ...).

indexing *{'xy', 'ij'}, default 'xy'* — 'xy' gives Cartesian (image) indexing; 'ij' gives matrix indexing.

sparse *bool, default False* — If True return sparse broadcastable grids to save memory.

copy *bool, default True* — If False return views where possible.

Returns: *list of ndarrays* — The coordinate matrices X, Y, ...

Example

```
X, Y = np.meshgrid(np.linspace(-2,2,100), np.linspace(-2,2,100))
Z = np.exp(-(X**2 + Y**2))
```

1.2 Shape and Rearrangement

```
np.reshape(a, newshape, order='C')
```

Give an array a new shape without changing its data. Equivalent method: `a.reshape(...)`. **Parameters**

a *array-like* — Array to reshape.

newshape *int or tuple of ints* — Target shape. One dimension may be -1, which is inferred from the array size.

order *{'C', 'F', 'A'}, default 'C'* — Order in which elements are read and placed.

Returns: *ndarray* — A reshaped view when possible, else a copy.

Example

```
v = a.reshape(-1)           # flatten to 1-D
M = a.reshape(3, -1)        # 3 rows, columns inferred
```

np.ravel(a, order='C')

Return a contiguous flattened 1-D array. Returns a view when possible (contrast `a.flatten()`, which always copies). **Parameters**

a *array-like* — Array to flatten.

order `{'C', 'F', 'A', 'K'}`, *default 'C'* — Reading order of the elements.

Returns: *ndarray* — Flattened array.

np.transpose(a, axes=None)

Permute the dimensions of an array. For 2-D this is the matrix transpose; the shorthand `a.T` does the same. **Parameters**

a *array-like* — Input array.

axes *tuple of ints, optional* — Permutation of the axis order. If omitted, the axes are reversed.

Returns: *ndarray* — A view with permuted axes.

np.concatenate((a1, a2, ...), axis=0)

Join a sequence of arrays along an existing axis. All arrays must match in shape except along that axis.

Parameters

(a1, a2, ...) *sequence of array-like* — The arrays to join, given as a tuple or list.

axis *int or None, default 0* — Axis to join along. If None, the arrays are flattened first.

Returns: *ndarray* — The concatenated array.

np.stack(arrays, axis=0)

Join a sequence of equally shaped arrays along a NEW axis, increasing the dimensionality by one. **Parameters**

arrays *sequence of array-like* — Arrays of identical shape.

axis *int, default 0* — Position of the new axis in the result.

Returns: *ndarray* — Array with one more dimension than each input.

np.vstack(tup) / np.hstack(tup)

Stack arrays vertically (row-wise) or horizontally (column-wise). Convenience wrappers around `concatenate`. **Parameters**

tup *sequence of array-like* — Arrays to stack. For `vstack` they must share column counts; for `hstack`, row counts.

Returns: *ndarray* — The stacked array.

np.split(ary, indices_or_sections, axis=0)

Split an array into multiple sub-arrays along an axis. **Parameters**

ary *ndarray* — Array to divide.

indices_or_sections *int or 1-D array* — If an int N, split into N equal parts (must divide evenly). If a list of indices, split at those positions.

axis *int, default 0* — Axis along which to split.

Returns: *list of ndarrays* — The sub-arrays.

np.expand_dims(a, axis)

Insert a new length-one axis at the given position, e.g. to turn a 1-D signal into a row or column vector for broadcasting. **Parameters**

a *array-like* — Input array.

axis *int or tuple of ints* — Position(s) where the new axis is placed.

Returns: *ndarray* — View with expanded shape.

np.squeeze(a, axis=None)

Remove length-one axes from the shape. **Parameters**

a *array-like* — Input array.

axis *int or tuple, optional* — Only squeeze the listed axes; error if any is not length one.

Returns: *ndarray* — View with the singleton dimensions removed.

np.repeat(a, repeats, axis=None)

Repeat each element of the array. Contrast np.tile, which repeats the whole array as a block. **Parameters**

a *array-like* — Input array.

repeats *int or array of ints* — How many times to repeat each element (may differ per element).

axis *int, optional* — Axis along which to repeat. If None, the array is flattened first.

Returns: *ndarray* — The repeated array.

np.tile(A, reps)

Construct a new array by tiling (block-repeating) A the given number of times along each axis. **Parameters**

A *array-like* — The tile to repeat.

reps *int or tuple of ints* — Number of repetitions along each axis.

Returns: *ndarray* — The tiled array.

np.roll(a, shift, axis=None)

Circularly shift array elements; values that roll past the end reappear at the start. Handy for a wrap-around (periodic) delay. **Parameters**

a *array-like* — Input array.

shift *int or tuple of ints* — Number of places to shift; positive shifts toward higher indices.

axis *int or tuple, optional* — Axis to shift along. If None the flattened array is shifted then reshaped.

Returns: *ndarray* — The shifted array.

np.pad(array, pad_width, mode='constant', **kwargs)

Pad an array along its edges. **Parameters**

array *array-like* — Array to pad.

pad_width *int or sequence* — Number of values padded before and after each axis, e.g. ((2,3),) or ((1,1),(2,2)).

mode *str, default 'constant'* — Padding strategy: 'constant', 'edge', 'reflect', 'wrap', 'mean', etc.

constant_values *scalar, optional* — Fill value when mode='constant' (default 0).

Returns: *ndarray* — The padded array.

```
np.flip(m, axis=None)
```

Reverse the order of elements along the given axis (time reversal for a 1-D signal is `np.flip(x)`). **Parameters**

m *array-like* — Input array.

axis *int or tuple, optional* — Axis to flip. If None, flip over all axes.

Returns: *ndarray* — A view with reversed order.

```
np.sort(a, axis=-1, kind=None, order=None)
```

Return a sorted copy of an array. **Parameters**

a *array-like* — Array to sort.

axis *int or None, default -1* — Axis to sort along; None flattens first.

kind {*'quicksort', 'mergesort', 'stable', 'heapsort'*}, *optional* — Sorting algorithm; 'stable' preserves order of ties.

order *str or list, optional* — For structured arrays, the field(s) to sort by.

Returns: *ndarray* — Sorted array.

```
np.argsort(a, axis=-1, kind=None)
```

Return the indices that would sort an array. Applying them (`a[idx]`) yields the sorted array. **Parameters**

a *array-like* — Array to sort.

axis *int or None, default -1* — Axis to sort along.

kind *str, optional* — Sorting algorithm.

Returns: *ndarray of int* — Index array.

1.3 Indexing, Searching, Selection

```
np.where(condition[, x, y])
```

With three arguments, return an array picking elements from `x` where `condition` is True and from `y` otherwise. With one argument, return the indices where `condition` is True. **Parameters**

condition *array-like of bool* — The selection mask.

x *array-like, optional* — Values chosen where `condition` is True.

y *array-like, optional* — Values chosen where `condition` is False.

Returns: *ndarray or tuple* — Selected values, or a tuple of index arrays if `x` and `y` are omitted.

Example

```
y = np.where(t >= 0, np.exp(-t), 0.0) # a causal signal
```

```
np.clip(a, a_min, a_max, out=None)
```

Limit the values of an array to the interval `[a_min, a_max]`; values outside are set to the nearest bound. **Parameters**

Parameters

a *array-like* — Array to clip.

a_min *scalar or array-like or None* — Lower bound (None means no lower clipping).

a_max *scalar or array-like or None* — Upper bound (None means no upper clipping).

out *ndarray, optional* — Array to write the result into.

Returns: *ndarray* — The clipped array.

np.nonzero(a)

Return the indices of the elements that are nonzero, as a tuple of arrays (one per dimension). **Parameters**

a *array-like* — Input array.

Returns: *tuple of ndarrays* — Index arrays locating the nonzero entries.

np.take(a, indices, axis=None)

Gather elements from an array at the given indices along an axis. **Parameters**

a *array-like* — Source array.

indices *array-like of int* — Indices of the values to extract.

axis *int, optional* — Axis to take from; None uses the flattened array.

Returns: *ndarray* — The gathered elements.

np.unique(ar, return_index=False, return_inverse=False, return_counts=False, axis=None)

Find the sorted unique elements of an array, optionally with bookkeeping arrays. **Parameters**

ar *array-like* — Input array.

return_index *bool, default False* — Also return indices of the first occurrences.

return_inverse *bool, default False* — Also return indices to reconstruct ar from the unique values.

return_counts *bool, default False* — Also return how many times each unique value appears.

axis *int, optional* — Operate along this axis instead of flattening.

Returns: *ndarray (and extras)* — The unique values, plus any requested index/count arrays.

1.4 Elementwise Math (Universal Functions)

These operate elementwise with broadcasting. Most ufuncs share the optional keyword arguments `out` (destination array) and `where` (a boolean mask selecting which elements to compute).

np.add / subtract / multiply / divide / power (x1, x2, /, out=None, where=True)

Elementwise arithmetic, equivalent to the operators `+` `-` `*` `/` `**`. Broadcasting aligns shapes. **Parameters**

x1, x2 *array-like* — The operands; broadcast against each other.

out *ndarray, optional* — Where to store the result.

where *array-like of bool, optional* — Compute only where True, leaving other out entries untouched.

Returns: *ndarray* — Elementwise result.

np.sqrt(x) / np.square(x) / np.cbrt(x)

Elementwise square root, square, and cube root. **Parameters**

x *array-like* — Input values (sqrt requires non-negative input for real output).

out *ndarray, optional* — Output array.

where *bool array, optional* — Selective computation mask.

Returns: *ndarray* — Transformed values.

np.exp(x) / np.log(x) / np.log10(x) / np.log2(x)

Elementwise exponential and logarithms. `np.expm1` and `np.log1p` give `exp(x)-1` and `log(1+x)` accurately for small `x`. **Parameters**

x *array-like* — Input values (log requires positive input).

out *ndarray, optional* — Output array.
where *bool array, optional* — Selective computation mask.
Returns: *ndarray* — Transformed values.
Example
`decay = np.exp(-0.5 * t)`

`np.sin / np.cos / np.tan / np.arctan2 (x, /, out=None)`

Elementwise trigonometric functions (arguments in radians). `np.arctan2(y, x)` returns the angle of the point (x, y) in the correct quadrant. **Parameters**

x *array-like* — Angle in radians (or y, x for `arctan2`).
out *ndarray, optional* — Output array.
Returns: *ndarray* — Trig values.

`np.abs(x)` (a.k.a. `np.absolute`)

Elementwise absolute value; for complex input it returns the magnitude. **Parameters**

x *array-like* — Input values.
Returns: *ndarray* — Magnitudes.

`np.sign(x)`

Elementwise sign: -1, 0, or +1 (0 maps to 0). Useful for building square waves. **Parameters**

x *array-like* — Input values.
Returns: *ndarray* — The signs.

`np.power(x1, x2)`

Raise each base in `x1` to the power in `x2`, elementwise with broadcasting. **Parameters**

x1 *array-like* — Bases.
x2 *array-like* — Exponents.
Returns: *ndarray* — `x1` raised to `x2`.

`np.round(a, decimals=0, out=None)`

Round to the given number of decimal places (banker's rounding at exact halves). **Parameters**

a *array-like* — Input values.
decimals *int, default 0* — Number of decimal places; negative rounds to tens, hundreds, etc.
out *ndarray, optional* — Output array.
Returns: *ndarray* — Rounded values.

`np.mod(x1, x2)` (a.k.a. `np.remainder`)

Elementwise remainder with the same sign as the divisor `x2`. Equivalent to the `%` operator. **Parameters**

x1 *array-like* — Dividend.
x2 *array-like* — Divisor.
Returns: *ndarray* — The remainders.
Example

```
phase = np.mod(t, 2*np.pi)  # wrap into one period
```

```
np.maximum(x1, x2) / np.minimum(x1, x2)
```

Elementwise maximum/minimum of two arrays (contrast `np.max`, which reduces a single array). **Parameters**

`x1, x2` *array-like* — Arrays compared elementwise.

Returns: *ndarray* — Elementwise extreme.

1.5 Reductions and Statistics

Reductions collapse one or more axes. They share the keywords `axis` (which axis/axes to reduce, `None` means all) and `keepdims` (retain reduced axes as length one for broadcasting).

```
np.sum(a, axis=None, dtype=None, keepdims=False, initial=0, where=True)
```

Sum of array elements over the given axis. **Parameters**

`a` *array-like* — Array to sum.

`axis` *int or tuple or None, default None* — Axis/axes to sum over; `None` sums everything.

`dtype` *data-type, optional* — Accumulator type; use `float64` to avoid integer overflow.

`keepdims` *bool, default False* — Keep reduced axes with length one.

`initial` *scalar, optional* — Starting value of the sum.

`where` *bool array, optional* — Include only these elements.

Returns: *ndarray or scalar* — The sum.

```
np.prod(a, axis=None, dtype=None, keepdims=False)
```

Product of array elements over an axis. Same `axis`/`dtype`/`keepdims` semantics as `np.sum`. **Parameters**

`a` *array-like* — Input array.

`axis` *int or tuple or None* — Axis to reduce.

`dtype` *data-type, optional* — Accumulator type.

`keepdims` *bool, default False* — Retain reduced dimensions.

Returns: *ndarray or scalar* — The product.

```
np.mean(a, axis=None, dtype=None, keepdims=False)
```

Arithmetic mean over the given axis. **Parameters**

`a` *array-like* — Input array.

`axis` *int or tuple or None* — Axis to average over.

`dtype` *data-type, optional* — Type used for the intermediate sum.

`keepdims` *bool, default False* — Retain reduced dimensions.

Returns: *ndarray or scalar* — The mean.

```
np.std(a, axis=None, dtype=None, ddof=0, keepdims=False)
```

Standard deviation over an axis. `np.var` is the variance with the same signature. **Parameters**

`a` *array-like* — Input array.

`axis` *int or tuple or None* — Axis to reduce.

`ddof` *int, default 0* — Delta degrees of freedom: the divisor is `N - ddof` (use 1 for the sample std).

`keepdims` *bool, default False* — Retain reduced dimensions.

Returns: *ndarray or scalar* — The standard deviation.

```
np.median(a, axis=None, keepdims=False)
```

Median value along an axis (average of the two central values for even counts). **Parameters**

a *array-like* — Input array.

axis *int or tuple or None* — Axis to reduce.

keepdims *bool, default False* — Retain reduced dimensions.

Returns: *ndarray or scalar* — The median.

```
np.percentile(a, q, axis=None, method='linear', keepdims=False)
```

Compute the q-th percentile of the data along an axis. **Parameters**

a *array-like* — Input array.

q *float or array of float in [0,100]* — Percentile(s) to compute (e.g. 50 for the median).

axis *int or tuple or None* — Axis to reduce.

method *str, default 'linear'* — Interpolation method between data points.

keepdims *bool, default False* — Retain reduced dimensions.

Returns: *scalar or ndarray* — The requested percentile(s).

```
np.min / np.max (a, axis=None, keepdims=False, initial, where)
```

Minimum/maximum value along an axis. (Also spelled np.amin / np.amax.) **Parameters**

a *array-like* — Input array.

axis *int or tuple or None* — Axis to reduce.

keepdims *bool, default False* — Retain reduced dimensions.

initial *scalar, optional* — A starting value that participates in the comparison.

Returns: *ndarray or scalar* — The extreme value.

```
np.argmin / np.argmax (a, axis=None)
```

Index of the minimum/maximum value along an axis (index into the flattened array if axis is None).

Parameters

a *array-like* — Input array.

axis *int, optional* — Axis along which to search.

Returns: *ndarray of int or int* — Location of the extreme.

```
np.cumsum(a, axis=None, dtype=None)
```

Cumulative sum of the elements along an axis. Scaling by the sample spacing turns this into a running (Riemann) integral. **Parameters**

a *array-like* — Input array.

axis *int, optional* — Axis to accumulate along; None flattens first.

dtype *data-type, optional* — Accumulator type.

Returns: *ndarray* — Array of running totals.

```
np.all / np.any (a, axis=None, keepdims=False)
```

Test whether all / any elements along an axis evaluate to True. **Parameters**

a *array-like* — Input array (usually boolean).
axis *int or tuple or None* — Axis to reduce.
keepdims *bool, default False* — Retain reduced dimensions.
Returns: *ndarray of bool or bool* — The logical reduction.

1.6 Calculus and Signal Processing

The numerical-analysis functions that recur throughout a signals course.

```
np.trapezoid(y, x=None, dx=1.0, axis=-1)
```

Integrate y along an axis using the composite trapezoidal rule. This is the renamed np.trapz (use np.trapz on NumPy < 2.0). The main tool for computing signal energy and power. **Parameters**

y *array-like* — Values to integrate.
x *array-like, optional* — Sample positions. If given, spacing may be non-uniform.
dx *scalar, default 1.0* — Uniform spacing between samples, used only when x is None.
axis *int, default -1* — Axis along which to integrate.

Returns: *float or ndarray* — The definite integral.

Example

```
E = np.trapezoid(np.abs(x)**2, t) # signal energy
```

```
np.gradient(f, *varargs, axis=None, edge_order=1)
```

Estimate the derivative of f using central differences in the interior and one-sided differences at the boundaries. **Parameters**

f *array-like* — Samples of the function to differentiate.
varargs *scalars or arrays, optional* — Sample spacing(s): a single dx, or coordinate arrays per axis.
axis *int or tuple, optional* — Axis/axes along which to differentiate (default all).
edge_order *{1, 2}, default 1* — Order of the one-sided difference used at the boundaries.

Returns: *ndarray or list of ndarrays* — The gradient(s).

```
np.diff(a, n=1, axis=-1, prepend=<no value>, append=<no value>)
```

Discrete difference along an axis: $\text{out}[i] = a[i+1] - a[i]$, applied n times. **Parameters**

a *array-like* — Input array.
n *int, default 1* — How many times the difference is taken.
axis *int, default -1* — Axis to difference along.
prepend / append *array-like, optional* — Values attached before/after along the axis before differencing (keeps the length).

Returns: *ndarray* — The n-th differences (length reduced by n unless padded).

```
np.interp(x, xp, fp, left=None, right=None, period=None)
```

One-dimensional piecewise-linear interpolation. Resamples a signal known at points xp onto new points x. **Parameters**

x *array-like* — Query coordinates where interpolated values are wanted.
xp *1-D array, increasing* — Known sample coordinates (must be increasing unless period is set).
fp *1-D array* — Known sample values at xp.

left float, optional — Value returned for $x \leq x_p[0]$ (default $x_p[0]$).
right float, optional — Value returned for $x \geq x_p[-1]$ (default $x_p[-1]$).
period float, optional — If set, treat the data as periodic with this period.

Returns: *ndarray* — Interpolated values at x .

Example

```
y_shifted = np.interp(t - t0, t, x, left=0, right=0) # delay by t0
```

`np.convolve(a, v, mode='full')`

Discrete linear convolution of two 1-D sequences (the operation an LTI filter performs). **Parameters**

a 1-D array — First sequence (e.g. the input signal).

v 1-D array — Second sequence (e.g. the impulse response).

mode {*'full', 'same', 'valid'*}, *default 'full'* — *'full'* returns the whole overlap ($\text{len } a + \text{len } v - 1$); *'same'* returns length $\max(\text{len } a, \text{len } v)$; *'valid'* returns only fully overlapping points.

Returns: *ndarray* — The convolution.

`np.correlate(a, v, mode='valid')`

Cross-correlation of two 1-D sequences; like convolution but without reversing the second sequence. **Parameters**

a, v 1-D arrays — The sequences to correlate.

mode {*'valid', 'same', 'full'*}, *default 'valid'* — Amount of overlap returned, as in `np.convolve`.

Returns: *ndarray* — The cross-correlation.

`np.fft.fft(a, n=None, axis=-1, norm=None)`

Compute the one-dimensional discrete Fourier transform of a sequence via the FFT. **Parameters**

a array-like — Input sequence (may be complex).

n int, optional — Length of the transformed axis: the input is zero-padded or truncated to n .

axis int, default -1 — Axis to transform.

norm {*None, 'ortho', 'forward', 'backward'*}, *optional* — Normalisation convention.

Returns: *complex ndarray* — The DFT coefficients.

`np.fft.fftfreq(n, d=1.0)`

Return the DFT sample frequencies (in cycles per unit) matching an n -point transform. **Parameters**

n int — Number of samples in the signal / transform.

d scalar, default 1.0 — Sample spacing (e.g. the time step). Frequencies scale as $1/d$.

Returns: *ndarray* — The frequency bins.

1.7 Linear Algebra

np.dot(a, b, out=None)

Dot product of two arrays: inner product for 1-D, matrix product for 2-D. For matrices prefer the @ operator or np.matmul. **Parameters**

a, b *array-like* — Operands whose inner dimensions must agree.

out *ndarray, optional* — Destination for the result.

Returns: *ndarray or scalar* — The dot/matrix product.

np.matmul(x1, x2) (the @ operator)

Matrix product with broadcasting over leading (batch) dimensions. Does not accept scalar operands. **Parameters**

x1, x2 *array-like* — Arrays whose last two dimensions are matrix-compatible.

Returns: *ndarray* — The matrix product.

np.linalg.solve(a, b)

Solve the linear system $a x = b$ for x . Numerically preferable to forming $\text{inv}(a) @ b$. **Parameters**

a *(..., M, M) array* — Square coefficient matrix (or a stack of them).

b *(..., M) or (..., M, K) array* — Right-hand side vector(s).

Returns: *ndarray* — The solution x .

Example

```
x = np.linalg.solve(A, b)
```

np.linalg.inv(a)

Compute the multiplicative inverse of a square matrix. Use solve instead when you only need to apply the inverse to a vector. **Parameters**

a *(..., M, M) array* — Invertible square matrix (or stack).

Returns: *ndarray* — The inverse matrix.

np.linalg.det(a)

Compute the determinant of a square matrix. **Parameters**

a *(..., M, M) array* — Square matrix (or stack).

Returns: *float or ndarray* — The determinant(s).

np.linalg.norm(x, ord=None, axis=None, keepdims=False)

Matrix or vector norm. **Parameters**

x *array-like* — Input vector or matrix.

ord *int, float, or str, optional* — Order of the norm: 2 (Euclidean, default for vectors), 1, np.inf, 'fro' (Frobenius), etc.

axis *int or 2-tuple, optional* — Axis/axes over which the norm is computed.

keepdims *bool, default False* — Retain reduced dimensions.

Returns: *float or ndarray* — The norm(s).

np.linalg.eig(a)

Compute the eigenvalues and right eigenvectors of a square matrix. **Parameters**

a (...) *M, M* array — Square matrix.

Returns: *tuple (eigvals, eigvecs)* — 1-D array of eigenvalues and a matrix whose columns are the eigenvectors.

np.linalg.svd(a, full_matrices=True, compute_uv=True)

Singular value decomposition $a = U S V^H$. **Parameters**

a (...) *M, N* array — Matrix to decompose.

full_matrices *bool, default True* — If True, U and V are full-sized; if False, reduced to economy size.

compute_uv *bool, default True* — If False, return only the singular values.

Returns: *tuple* — U, the singular values S, and V^H (or just S).

1.8 Logic, Comparison, Complex

np.isclose(a, b, rtol=1e-05, atol=1e-08, equal_nan=False)

Elementwise test of approximate equality within a relative and absolute tolerance. **Parameters**

a, b *array-like* — Arrays to compare.

rtol *float, default 1e-5* — Relative tolerance.

atol *float, default 1e-8* — Absolute tolerance (dominates near zero).

equal_nan *bool, default False* — If True, treat NaNs in the same position as equal.

Returns: *ndarray of bool* — Elementwise closeness.

np.allclose(a, b, rtol=1e-05, atol=1e-08, equal_nan=False)

Return a single True/False for whether all elements of a and b are close. Ideal for verifying that two computations agree. **Parameters**

a, b *array-like* — Arrays to compare.

rtol *float, default 1e-5* — Relative tolerance.

atol *float, default 1e-8* — Absolute tolerance.

equal_nan *bool, default False* — Treat matching NaNs as equal.

Returns: *bool* — Whether the arrays match within tolerance.

np.angle(z, deg=False) / np.conj(z)

np.angle returns the phase (argument) of complex values; **np.conj** returns the complex conjugate. Combined with **np.abs** they cover polar form. **Parameters**

z *array-like* — Complex (or real) input.

deg *bool, default False* — For **np.angle**, return the phase in degrees instead of radians.

Returns: *ndarray* — The phase, or the conjugate.

1.9 Random Sampling (Generator API)

The modern interface: create a Generator with `np.random.default_rng(seed)`, then call its methods. Passing a seed makes results reproducible.

```
np.random.default_rng(seed=None)
```

Construct a random-number Generator, the recommended entry point for random sampling. **Parameters**
seed *int, sequence, or None* — Seed for reproducibility. None draws fresh entropy from the OS.

Returns: *Generator* — An object exposing the sampling methods below.

Example

```
rng = np.random.default_rng(42)
```

```
rng.random(size=None, dtype=np.float64)
```

Draw samples from the uniform distribution on the half-open interval $[0, 1)$. **Parameters**

size *int or tuple, optional* — Output shape. None returns a single scalar.

dtype *data-type, default float64* — Floating type of the samples.

Returns: *float or ndarray* — Uniform samples in $[0, 1)$.

```
rng.normal(loc=0.0, scale=1.0, size=None)
```

Draw samples from a Gaussian (normal) distribution. **Parameters**

loc *float or array-like, default 0.0* — Mean of the distribution.

scale *float or array-like, default 1.0* — Standard deviation (must be non-negative).

size *int or tuple, optional* — Output shape.

Returns: *float or ndarray* — Normal samples.

```
rng.uniform(low=0.0, high=1.0, size=None)
```

Draw samples from a uniform distribution over $[low, high)$. **Parameters**

low *float or array-like, default 0.0* — Lower bound.

high *float or array-like, default 1.0* — Upper bound (excluded).

size *int or tuple, optional* — Output shape.

Returns: *float or ndarray* — Uniform samples.

```
rng.integers(low, high=None, size=None, endpoint=False)
```

Draw random integers from low (inclusive) to high (exclusive by default). **Parameters**

low *int* — Lowest integer, or the exclusive high if high is None.

high *int, optional* — Upper bound.

size *int or tuple, optional* — Output shape.

endpoint *bool, default False* — If True, high becomes inclusive.

Returns: *int or ndarray* — Random integers.

```
rng.choice(a, size=None, replace=True, p=None)
```

Draw a random sample from a 1-D array or from `range(a)`. **Parameters**

a *1-D array-like or int* — The population to sample from (or its length).

size *int or tuple, optional* — Number/shape of samples.

replace *bool, default True* — Whether sampling is with replacement.

p *1-D array-like, optional* — Per-element selection probabilities (must sum to 1).

Returns: *scalar or ndarray* — The drawn sample(s).

2 Matplotlib

2.1 Figures and Axes

A Figure is the whole canvas; an Axes is one plotting area (with its x/y axes) inside it. The object-oriented style – create a Figure and Axes, then call methods on the Axes – is recommended over the implicit `plt.*` state machine.

```
plt.subplots(nrows=1, ncols=1, figsize=None, sharex=False, sharey=False, squeeze=True, dpi=None)
```

Create a Figure and a grid of Axes in one call. The usual starting point for a plot. **Parameters**

nrows, ncols *int, default 1* — Number of rows and columns in the Axes grid.

figsize (*float, float*), *optional* — Figure size in inches, (width, height).

sharex *bool or {'none', 'all', 'row', 'col'}, default False* — Share the x-axis range/ticks across subplots.

sharey *bool or str, default False* — Share the y-axis similarly.

squeeze *bool, default True* — If True, collapse the returned Axes array when possible (a single Axes is returned bare).

dpi *float, optional* — Resolution in dots per inch.

Returns: (*Figure, Axes or ndarray of Axes*) — The figure and its Axes.

Example

```
fig, ax = plt.subplots(figsize=(8, 4))
fig, axs = plt.subplots(2, 2, sharex=True)
```

```
plt.figure(num=None, figsize=None, dpi=None, facecolor=None)
```

Create a new Figure (or activate an existing one). Use when you want the figure before adding axes manually. **Parameters**

num *int or str, optional* — Identifier of the figure; reuses it if it already exists.

figsize (*float, float*), *optional* — Size in inches (width, height).

dpi *float, optional* — Resolution in dots per inch.

facecolor *color, optional* — Background colour of the figure.

Returns: *Figure* — The new or activated figure.

```
fig.add_subplot(nrows, ncols, index, projection=None)
```

Add a single Axes to a figure at a position in an implied grid. **Parameters**

nrows, ncols *int* — Shape of the subplot grid.

index *int* — 1-based position of this Axes, counting left-to-right, top-to-bottom.

projection *str, optional* — Axes projection, e.g. 'polar' or '3d'.

Returns: *Axes* — The newly added Axes.

```
ax.twinx() / ax.twiny()
```

Create a second Axes that shares the x-axis (twinx) or y-axis (twiny) with ax, enabling a second independent y (or x) scale on the opposite side. **Returns:** *Axes* — The twin Axes.

```
fig.tight_layout(pad=1.08, h_pad=None, w_pad=None)
```

Automatically adjust subplot spacing so labels, ticks, and titles do not overlap or get clipped. **Parameters**

pad *float, default 1.08* — Padding around the figure edge, in fractions of the font size.

h_pad, w_pad *float, optional* — Vertical/horizontal padding between adjacent subplots.

Returns: *None* — Adjusts the layout in place.

```
fig.suptitle(t, fontsize=None, y=0.98)
```

Add a centred title spanning the whole figure (above all subplots). **Parameters**

t *str* — The title text.

fontsize *int or str, optional* — Font size of the title.

y *float, default 0.98* — Vertical position in figure coordinates.

Returns: *Text* — The created text object.

2.2 Plotting: Lines and Points

The most common Axes plotting methods. Line-style keywords (color, linestyle, linewidth, marker, markersize, alpha, label, zorder) are accepted by most of them.

```
ax.plot(x, y, [fmt], color=None, linestyle='-', linewidth=None, marker=None, markersize=None, label=None, alpha=None)
```

Plot y versus x as connected line segments and/or markers. With a single argument, x defaults to the sample index. **Parameters**

x, y *array-like* — Data coordinates. y alone plots against 0..N-1.

fmt *str, optional* — Compact style string combining colour, line style and marker, e.g. 'r-o'.

color *color, optional* — Line colour ('C0'..'C9', names, hex, or grey levels like '0.6').

linestyle *str, default '-'* — '-', '-', '-', '-', or '-'.

linewidth *float, optional* — Line width in points.

marker *str, optional* — Marker symbol, e.g. 'o', '.', 'x', 's', '^'.

markersize *float, optional* — Marker size in points.

label *str, optional* — Legend label for this line.

alpha *float in [0,1], optional* — Opacity (1 is opaque).

Returns: *list of Line2D* — The plotted line object(s).

Example

```
ax.plot(t, x, color='C0', linewidth=1.5, label='x(t)')
```

```
ax.scatter(x, y, s=None, c=None, marker='o', cmap=None, alpha=None, edgecolors=None)
```

Draw a scatter plot of points, optionally with per-point size and colour. **Parameters**

x, y *array-like* — Point coordinates.

s *scalar or array, optional* — Marker area(s) in points squared.

c *color or array, optional* — Point colour(s); an array is mapped through cmap.

marker *str, default 'o'* — Marker style.

cmap *str or Colormap, optional* — Colormap used when c is numeric.

alpha *float, optional* — Opacity.

edgecolors *color, optional* — Colour of the marker edges.

Returns: *PathCollection* — The scatter collection.

```
ax.stem(x, y, linefmt=None, markerfmt=None, basefmt=None)
```

Draw a stem (lollipop) plot: a vertical line to each point from a baseline. The natural way to display discrete-time signals. **Parameters**

x *array-like* — Sample positions (index n).

y *array-like* — Sample values x[n].

linefmt *str, optional* — Style of the stem lines.

markerfmt *str, optional* — Style of the head markers.

basefmt *str, optional* — Style of the baseline.

Returns: *StemContainer* — The stem artists.

`ax.step(x, y, where='pre')`

Draw a piecewise-constant (staircase) line, appropriate for sampled or held signals. **Parameters**

`x, y` *array-like* — Data coordinates.

`where` {'pre', 'post', 'mid'}, *default* 'pre' — Where the step occurs relative to the samples.

Returns: *list of Line2D* — The step line(s).

`ax.fill_between(x, y1, y2=0, where=None, interpolate=False, alpha=None, color=None)`

Fill the area between two horizontal curves `y1` and `y2`. **Parameters**

`x` *array-like* — x coordinates shared by both curves.

`y1` *array-like* — First curve.

`y2` *array-like or scalar, default 0* — Second curve (a constant such as 0 is common).

`where` *bool array, optional* — Fill only where this condition is True.

`interpolate` *bool, default False* — Interpolate crossing points for a clean fill boundary.

`alpha` *float, optional* — Opacity of the fill.

`color` *color, optional* — Fill colour.

Returns: *PolyCollection* — The filled region.

`ax.errorbar(x, y, yerr=None, xerr=None, fmt='', ecolor=None, capsize=None)`

Plot points or a line with error bars. **Parameters**

`x, y` *array-like* — Data coordinates.

`yerr` *scalar or array, optional* — Vertical error magnitudes (symmetric, or a 2xN array for asymmetric).

`xerr` *scalar or array, optional* — Horizontal error magnitudes.

`fmt` *str, optional* — Line/marker format for the data points; " draws error bars only.

`ecolor` *color, optional* — Colour of the error bars.

`capsize` *float, optional* — Length of the error-bar caps in points.

Returns: *ErrorbarContainer* — The line, caps, and bars.

`ax.loglog / ax.semilogx / ax.semilogy (x, y, **kwargs)`

Plot with logarithmic scaling on both axes, on x only, or on y only. Accepts the same keywords as `ax.plot`.

Parameters

`x, y` *array-like* — Data coordinates (positive where a log axis applies).

Returns: *list of Line2D* — The plotted line(s).

2.3 Plotting: Bars, Histograms, Images

`ax.bar(x, height, width=0.8, bottom=None, align='center', color=None)`

Draw a vertical bar chart (`ax.barh` is the horizontal version, taking `y` and `width`). **Parameters**

`x` *array-like* — x positions of the bar centres (or left edges).

`height` *array-like* — Heights of the bars.

`width` *scalar or array, default 0.8* — Bar width(s).

`bottom` *scalar or array, optional* — Baseline of the bars (for stacking).

`align` {'center', 'edge'}, *default* 'center' — Whether `x` gives the centre or the left edge.

color *color or list, optional* — Bar colour(s).

Returns: *BarContainer* — The bar rectangles.

```
ax.hist(x, bins=10, range=None, density=False, cumulative=False, histtype='bar', color=None)
```

Compute and draw a histogram of *x*. **Parameters**

x *array-like* — Data to bin.

bins *int or sequence, default 10* — Number of equal-width bins, or explicit bin edges.

range *(float, float), optional* — Lower and upper range of the bins.

density *bool, default False* — If True, normalise so the histogram integrates to 1.

cumulative *bool, default False* — If True, accumulate counts.

histtype *{'bar', 'step', 'stepfilled'}, default 'bar'* — Visual style.

color *color, optional* — Bar colour.

Returns: *tuple (n, bins, patches)* — Counts, bin edges, and the drawn patches.

```
ax.imshow(X, cmap=None, aspect=None, interpolation=None, origin=None, extent=None, vmin=None, vmax=None)
```

Display 2-D array data as an image (heatmap). Ideal for spectrograms and matrices. **Parameters**

X *array-like* — 2-D (grayscale) or 3-D (RGB/RGBA) image data.

cmap *str or Colormap, optional* — Colormap for scalar data, e.g. 'viridis', 'gray'.

aspect *{'equal', 'auto'} or float, optional* — Pixel aspect ratio; 'auto' fills the Axes.

interpolation *str, optional* — Resampling method, e.g. 'nearest', 'bilinear'.

origin *{'upper', 'lower'}, optional* — Whether row 0 is at the top or bottom.

extent *(left, right, bottom, top), optional* — Data coordinates of the image corners.

vmin, vmax *float, optional* — Data values mapped to the ends of the colormap.

Returns: *AxesImage* — The image artist.

```
ax.pcolormesh(X, Y, C, shading='flat', cmap=None, vmin=None, vmax=None)
```

Draw a (possibly non-uniform) quadrilateral mesh coloured by *C*; more flexible than *imshow* for irregular grids. **Parameters**

X, Y *array-like* — Coordinates of the mesh vertices.

C *array-like* — Values that colour each cell.

shading *{'flat', 'nearest', 'gouraud', 'auto'}, default 'flat'* — How cells are coloured/interpolated.

cmap *str or Colormap, optional* — Colormap.

vmin, vmax *float, optional* — Colour scale limits.

Returns: *QuadMesh* — The mesh artist.

```
ax.contour / ax.contourf (X, Y, Z, levels=None, cmap=None)
```

Draw contour lines (contour) or filled contour regions (contourf) of a surface *Z* over the grid (*X, Y*). **Parameters**

X, Y *array-like* — Grid coordinates (as from *np.meshgrid*).

Z *array-like* — Surface height at each grid point.

levels *int or array, optional* — Number of contour levels, or the explicit level values.

cmap *str or Colormap, optional* — Colormap for the contours.

Returns: *QuadContourSet* — The contour set (pass to *colorbar*).

```
fig.colorbar(mappable, ax=None, label=None, orientation='vertical')
```

Add a colorbar that explains the colour-to-value mapping of an image, mesh, or contour set. **Parameters**

mappable *ScalarMappable* — The object returned by imshow/pcolormesh/scatter/contourf.

ax *Axes or list, optional* — Axes to steal space from for the colorbar.

label *str, optional* — Label drawn alongside the colorbar.

orientation *{'vertical','horizontal'}, default 'vertical'* — Colorbar orientation.

Returns: *Colorbar* — The colorbar object.

2.4 Axis Configuration

```
ax.set_xlabel(xlabel, fontsize=None, labelpad=None) / ax.set_ylabel(...) / ax.set_title(...)
```

Set the x-axis label, y-axis label, or the Axes title. **Parameters**

xlabel / ylabel / label *str* — The text to display.

fontsize *int or str, optional* — Font size.

labelpad *float, optional* — Spacing between the label and the axis, in points.

Returns: *Text* — The label/title text object.

```
ax.set_xlim(left=None, right=None) / ax.set_ylim(bottom=None, top=None)
```

Set the data limits (visible range) of an axis. Passing a reversed pair inverts the axis. **Parameters**

left / bottom *float, optional* — Lower data limit.

right / top *float, optional* — Upper data limit.

Returns: *tuple* — The new limits.

```
ax.set_xticks(ticks, labels=None, minor=False) / ax.set_yticks(...)
```

Set the tick locations (and optionally their text labels) on an axis. **Parameters**

ticks *array-like* — Positions at which to place ticks.

labels *list of str, optional* — Custom text for each tick.

minor *bool, default False* — Whether these are minor rather than major ticks.

Returns: *list* — The tick objects.

```
ax.tick_params(axis='both', which='major', labelsize=None, direction=None, length=None, rotation=None)
```

Change the appearance of ticks and their labels. **Parameters**

axis *{'x','y','both'}, default 'both'* — Which axis to modify.

which *{'major','minor','both'}, default 'major'* — Which set of ticks.

labelsize *float or str, optional* — Font size of the tick labels.

direction *{'in','out','inout'}, optional* — Whether ticks point inward or outward.

length *float, optional* — Tick length in points.

rotation *float, optional* — Rotation angle of the tick labels.

Returns: *None* — Modifies the ticks in place.

```
ax.grid(visible=True, which='major', axis='both', color=None, linestyle=None, alpha=None)
```

Toggle and style the background grid lines. **Parameters**

visible *bool, default True* — Whether to show the grid.

which {*'major', 'minor', 'both'*}, *default 'major'* — Which ticks get grid lines.

axis {*'x', 'y', 'both'*}, *default 'both'* — Which direction(s) to draw.

color *color, optional* — Grid line colour.

linestyle *str, optional* — Grid line style, e.g. *'.'*.

alpha *float, optional* — Grid opacity.

Returns: *None* — Modifies the grid in place.

```
ax.axhline(y=0, xmin=0, xmax=1, color=None, linestyle='-', linewidth=None) / ax.axvline(x=0, ...)
```

Draw a horizontal (axhline) or vertical (axvline) reference line spanning the Axes. **Parameters**

y / x *float, default 0* — Position of the line in data coordinates.

xmin, xmax / ymin, ymax *float in [0,1], default 0..1* — Fractional extent of the line across the Axes.

color *color, optional* — Line colour.

linestyle *str, default '-'* — Line style.

linewidth *float, optional* — Line width.

Returns: *Line2D* — The reference line.

```
ax.axvspan(xmin, xmax, ymin=0, ymax=1, color=None, alpha=None) / ax.axhspan(...)
```

Shade a vertical (axvspan) or horizontal (axhspan) band across the Axes – e.g. to highlight one period.

Parameters

xmin, xmax *float* — Data-coordinate edges of the band.

ymin, ymax *float in [0,1], default 0..1* — Fractional vertical extent.

color *color, optional* — Fill colour.

alpha *float, optional* — Opacity of the shaded band.

Returns: *Polygon* — The shaded patch.

```
ax.set_xscale / ax.set_yscale ('log', base=10)
```

Set the scale of an axis to 'linear', 'log', 'symlog', or 'logit'. **Parameters**

value *str* — The scale name, e.g. *'log'*.

base *float, optional* — Logarithm base for a log scale.

Returns: *None* — Changes the scale in place.

```
ax.set_aspect(aspect, adjustable=None)
```

Set the aspect ratio of the Axes ('equal' makes one data unit equal in x and y). **Parameters**

aspect {*'auto', 'equal'*} *or float* — Ratio of y-unit to x-unit length.

adjustable {*'box', 'datalim'*}, *optional* — What is adjusted to achieve the aspect.

Returns: *None* — Applies the aspect in place.

2.5 Legends, Text, Annotations

```
ax.legend(handles=None, labels=None, loc='best', ncol=1, fontsize=None, frameon=True, title=None)
```

Add a legend describing the labelled artists on the Axes. **Parameters**

handles *list of Artist, optional* — Artists to include; by default all labelled artists are used.

labels *list of str, optional* — Legend entry texts (override the artists' labels).

loc *str or int, default 'best'* — Placement, e.g. 'upper right', 'lower left', 'best'.

ncol *int, default 1* — Number of columns in the legend.

fontsize *int or str, optional* — Entry font size.

frameon *bool, default True* — Whether to draw the legend's frame.

title *str, optional* — A title for the legend box.

Returns: *Legend* — The legend object.

```
ax.text(x, y, s, fontsize=None, ha='left', va='baseline', color=None, rotation=0)
```

Place arbitrary text at data coordinates (x, y). **Parameters**

x, y *float* — Position of the text in data coordinates.

s *str* — The text string.

fontsize *int or str, optional* — Font size.

ha { 'left', 'center', 'right' }, *default 'left'* — Horizontal alignment relative to (x, y).

va { 'top', 'center', 'bottom', 'baseline' }, *default 'baseline'* — Vertical alignment.

color *color, optional* — Text colour.

rotation *float, default 0* — Rotation angle in degrees.

Returns: *Text* — The text object.

```
ax.annotate(text, xy, xytext=None, arrowprops=None, ha='left', fontsize=None)
```

Annotate a point xy with text, optionally drawing an arrow from the text to the point. **Parameters**

text *str* — The annotation string.

xy (*float, float*) — The point being annotated, in data coordinates.

xytext (*float, float*), *optional* — Where to place the text (defaults to xy).

arrowprops *dict, optional* — Arrow style, e.g. dict(arrowstyle='->').

ha { 'left', 'center', 'right' }, *optional* — Horizontal alignment of the text.

fontsize *int or str, optional* — Font size.

Returns: *Annotation* — The annotation object.

Example

```
ax.annotate('peak', xy=(x0, y0), xytext=(x0+1, y0+1),
           arrowprops=dict(arrowstyle='->'))
```

2.6 Saving, Display, Global Style

```
fig.savefig(fname, dpi=None, bbox_inches=None, transparent=False, facecolor='auto', format=None)
```

Save the figure to a file. The format is inferred from the file extension unless given explicitly. **Parameters**

fname *str or path* — Output path, e.g. 'plot.png' or 'fig.pdf'.

dpi *float, optional* — Resolution in dots per inch (raster formats).

bbox_inches *str or Bbox, optional* — 'tight' trims surrounding whitespace.

transparent *bool, default False* — Make the figure background transparent.

facecolor *color, optional* — Background colour of the saved figure.

format *str, optional* — Explicit format, e.g. 'png', 'pdf', 'svg'.

Returns: *None* — Writes the file.

Example

```
fig.savefig('out.png', dpi=150, bbox_inches='tight')
```

```
plt.show()
```

Display all open figures and (in scripts) block until the windows are closed. Not needed in notebooks with inline plotting. **Returns:** *None* — Renders the figures.

```
plt.close(fig=None)
```

Close a figure window and free its memory. Important in loops that create many figures. **Parameters**

fig *Figure, int, str, or 'all', optional* — Which figure to close; 'all' closes every open figure.

Returns: *None* — Closes the figure(s).

```
matplotlib.use(backend)
```

Select the rendering backend. Call before importing pyplot. Use 'Agg' for headless scripts/servers that only save files. **Parameters**

backend *str* — Backend name, e.g. 'Agg', 'TkAgg', 'QtAgg'.

Returns: *None* — Sets the backend.

Example

```
import matplotlib
matplotlib.use('Agg')      # no display; savefig only
import matplotlib.pyplot as plt
```

```
plt.style.use(style)
```

Apply a predefined style sheet that changes default colours, fonts, and grids for subsequent plots. **Parameters**

style *str, dict, or list* — A style name (e.g. 'ggplot', 'seaborn-v0.8'), a settings dict, or a list combining several.

Returns: *None* — Updates the defaults.

```
plt.rcParams[key] = value
```

Read or set a global default (a 'runtime configuration' parameter) affecting all subsequent plots. **Parameters**

key *str* — The parameter name, e.g. 'font.size', 'lines.linewidth', 'figure.dpi'.

value *various* — The new default value.

Returns: *None* — Mutates the global configuration.

Example

```
plt.rcParams['font.size'] = 11
```